



Escuela Politécnica Nacional

Nombre del Estudiante: Alexis Sotomayor

Fecha: 22/01/2026

Prueba 002 II Bimestre

 Proyecto 001 - Documentación de Endpoints con Swagger

 Objetivo

Que los estudiantes tomen los endpoints RESTful implementados en el examen (NestJS + TypeORM + SQLite, relación 1 a muchos) y los documenten automáticamente con **Swagger (OpenAPI)** dentro del mismo proyecto.

 Instrucciones para los estudiantes

1. Repositorio

- Usar el mismo repositorio del curso en la organización de GitHub.
- Crear una carpeta llamada Proyecto-001.
- Subir ahí el código fuente del proyecto con Swagger configurado.

2. Instalación de Swagger en NestJS

- Instalar dependencias:
- Configurar Swagger en main.ts:

3. Decoradores en los Endpoints

- Usar los decoradores de @nestjs/swagger para documentar cada controlador y DTO.
- Ejemplo en teams.controller.ts:
- Ejemplo en DTO:

4. Endpoints a Documentar

- **Teams**
 - GET /teams
 - GET /teams/:id
 - POST /teams
 - PUT /teams/:id
 - DELETE /teams/:id
- **Players**
 - GET /players
 - GET /players/:id
 - POST /players
 - PUT /players/:id
 - DELETE /players/:id
 - GET /teams/:id/players

5. README.md

- Instrucciones para instalar dependencias (npm install).
- Cómo correr el servidor (npm run start:dev).
- Cómo acceder a Swagger (<http://localhost:3000/api>).
- Ejemplo de endpoints documentados.

Criterios de Evaluación

- Proyecto correctamente subido al repositorio del curso.
- Swagger instalado y configurado en main.ts.
- Endpoints documentados con decoradores (@ApiTags, @ApiOperation, @ApiResponse).
- DTOs documentados con @ApiProperty.
- Documentación accesible en /api.



- README claro y completo.

Desarrollo de la practica

El desarrollo de esta práctica se centró en la implementación y documentación de una API RESTful utilizando NestJS, TypeORM y SQLite, con el objetivo principal de integrar Open API (Swagger) para la documentación automática.

Los pasos realizados fueron:

1. Configuración del Entorno:

- Se inicializó un nuevo proyecto NestJS (`Proyecto-001`).
- Se instalaron las dependencias necesarias: `@nestjs/typeorm`, `typeorm`, `sqlite3` para la persistencia de datos, y `@nestjs/swagger`, `swagger-ui-express` para la documentación.

2. Modelado de Datos (Entities):

- Se definieron dos entidades principales: `Team` (Equipo) y `Player` (Jugador).
- Se estableció una relación **uno a muchos (1:N)**: Un equipo tiene muchos jugadores, y un jugador pertenece a un equipo.
- Se utilizaron decoradores de TypeORM (`@Entity`, `@Column`, `@OneToMany`, `@ManyToOne`) para mapear estas estructuras a la base de datos SQLite.

3. Implementación de la Lógica (Services & Controllers):

- Se generaron módulos, controladores y servicios para `Teams` y `Players`.
- Se implementaron los métodos CRUD (Create, Read, Update, Delete) en los servicios, interactuando con la base de datos a través de Repositorios.
- Se creó un endpoint específico `GET /teams/:id/players` para consultar la relación.

4. Integración de Swagger:

- Se configuró el `DocumentBuilder` en el archivo `main.ts` para inicializar



Swagger en la ruta `/api`.

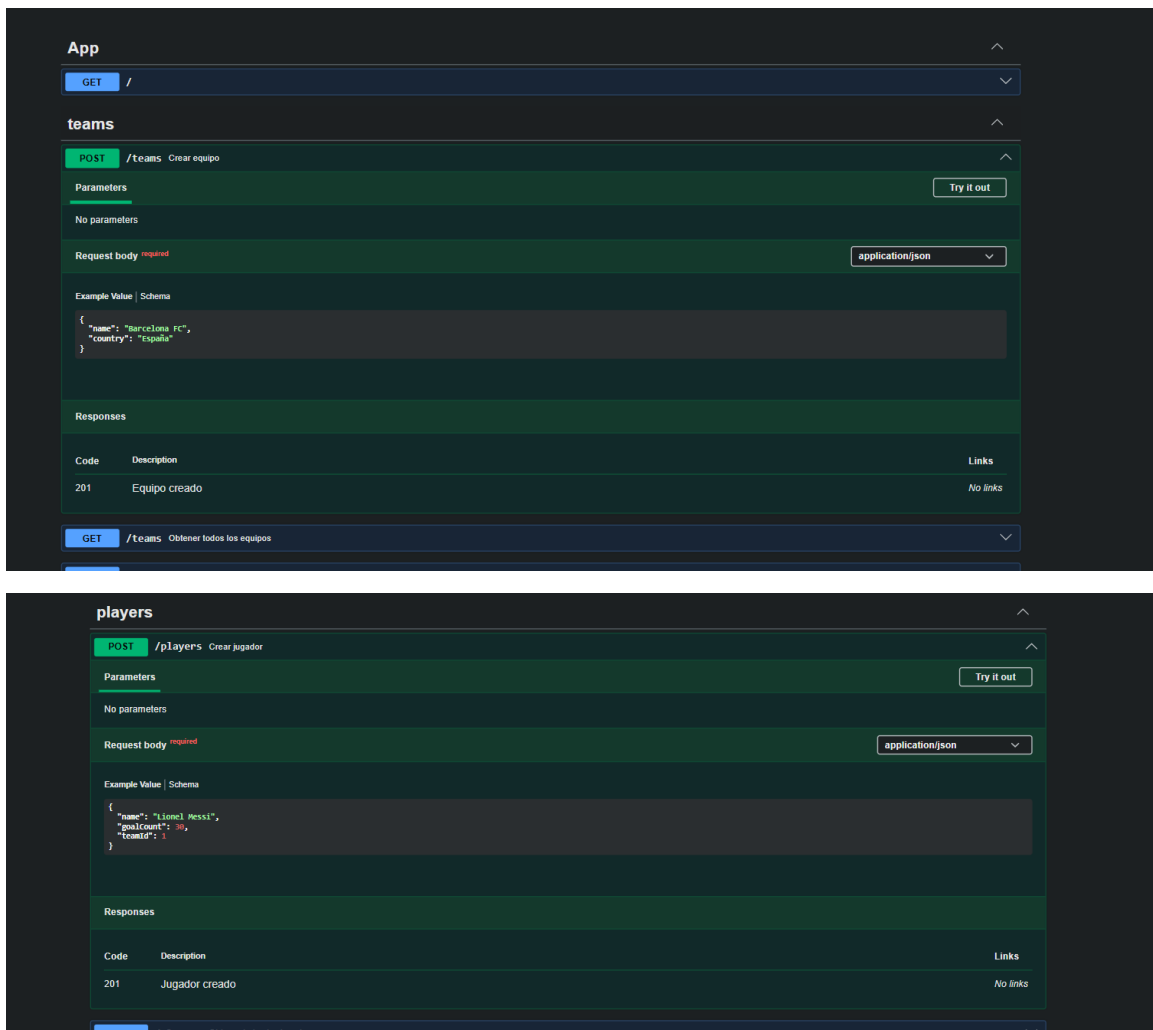
- Se decoraron los Controladores con `@ApiTags` para agrupar los endpoints.
- Se decoraron los métodos con `@ApiOperation` (descripción) y `@ApiResponse` (códigos de estado esperados).
- Se documentaron los DTOs (`CreateTeamDto`, `CreatePlayerDto`) usando `@ApiModelProperty` para mostrar ejemplos de los datos requeridos en la interfaz gráfica.

5. Validación:

- Se añadieron validaciones globales (`ValidationPipe`) y decoradores `class-validator` para asegurar la integridad de los datos de entrada.

Implementación de la documentación de la API





The image displays two screenshots of the Swagger UI interface. The top screenshot shows the 'App' section with a 'GET /' endpoint and a 'POST /teams' endpoint. The 'POST /teams' endpoint is selected, showing its parameters (none), request body (required, application/json), and an example value:

```
{ "name": "barcelona FC", "country": "España" }
```

. The bottom screenshot shows the 'players' section with a 'POST /players' endpoint. The 'POST /players' endpoint is selected, showing its parameters (none), request body (required, application/json), and an example value:

```
{ "name": "Lionel Messi", "goals": 10, "teamId": 1 }
```

. Both endpoints show a response code of 201 with the description 'Equipo creado' or 'Jugador creado'.

Análisis de Resultados

Tras la implementación, se realizaron pruebas funcionales accediendo a la interfaz de Swagger UI en `http://localhost:3000/api`.

- **Accesibilidad:** La interfaz de usuario de Swagger se cargó correctamente, mostrando todos los endpoints agrupados por "teams" y "players".
- **Funcionalidad CRUD:** Se verificó la correcta creación, lectura, actualización y eliminación de registros directamente desde el navegador.
 - Al crear un equipo, el servidor respondió con código `201 Created`.
 - Al consultar el listado, se obtuvo un JSON con la estructura esperada.
- **Relaciones:** Se confirmó que al crear jugadores asociados a un `teamId`, la

base de datos respeta la relación foránea. El endpoint ``/teams/:id/players`` devolvió correctamente la lista de jugadores pertenecientes a ese equipo.

- **Documentación Interactiva:** Los ejemplos definidos en los DTOs aparecieron precargados en los formularios de prueba, facilitando el envío de peticiones sin necesidad de escribir el JSON manualmente.

Conclusiones:

- La integración de Swagger en NestJS permite generar documentación viva y siempre actualizada con un esfuerzo mínimo, utilizando decoradores directamente en el código fuente. Esto elimina la discrepancia entre la documentación y la implementación real.
- Swagger UI actúa como una herramienta potente para pruebas rápidas, reemplazando en muchos casos la necesidad de herramientas externas como Postman para verificaciones básicas, lo que agiliza el ciclo de desarrollo.
- El uso de DTOs junto con los decoradores de API proporciona una definición clara de los contratos de la API (Request/Response), lo cual es crucial para que otros desarrolladores o equipos de frontend puedan consumir los servicios sin ambigüedades.

NOTA: La implementación de los ejemplos del consumo del proyecto para el Backend para las rutas de Teams y Players